

2024 DL Lab2 Super Resolution

● Task 1

```

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super(ResidualBlock, self).__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.prelu = nn.PReLU()
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

    def forward(self, x):
        residual = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.prelu(x)
        x = self.conv2(x)
        x = self.bn2(x)
        return x + residual

class SRResNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3, num_residual_blocks=16):
        super(SRResNet, self).__init__()
        # First conv layer
        self.conv1 = nn.Conv2d(in_channels, 64, kernel_size=9, stride=1, padding=4)
        self.prelu = nn.PReLU()

        # Residual blocks
        self.residual_blocks = nn.Sequential(
            *[ResidualBlock(64) for _ in range(num_residual_blocks)]
        )

        # Second conv layer after residual blocks
        self.conv2 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(64)

        # Upsampling layers
        self.upsample1 = nn.Conv2d(64, 256, kernel_size=3, stride=1, padding=1)
        self.pixel_shuffle1 = nn.PixelShuffle(2)
        self.prelu1 = nn.PReLU()

        self.upsample2 = nn.Conv2d(64, 256, kernel_size=3, stride=1, padding=1)
        self.pixel_shuffle2 = nn.PixelShuffle(2)
        self.prelu2 = nn.PReLU()

        # Final output conv layer
        self.conv3 = nn.Conv2d(64, out_channels, kernel_size=9, stride=1, padding=4)

```

```

def forward(self, x):
    # First conv + PReLU
    x = self.prelu(self.conv1(x))

    # Save the residual for skip connection
    residual = x

    # Residual blocks
    x = self.residual_blocks(x)

    # Second conv + BatchNorm (optional) + skip connection
    x = self.conv2(x)
    x = self.bn1(x) # 使用 BatchNorm
    x = x + residual # Skip connection

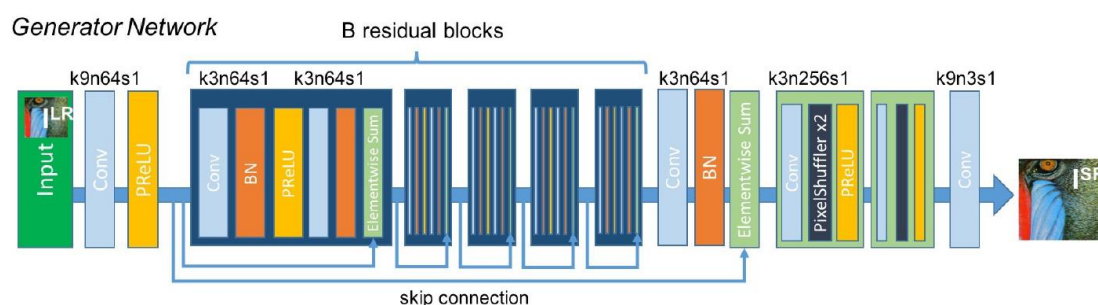
    # Upsampling + PixelShuffle + PReLU
    x = self.pixel_shuffle1(self.upsample1(x))
    x = self.prelu1(x)
    x = self.pixel_shuffle2(self.upsample2(x))
    x = self.prelu2(x)

    # Final conv to get the output image
    x = self.conv3(x)

    return x

```

上述的程式碼架構 SRResNet 是根據助教所提供的 Network，所刻出來的。



SRResNet 網路架構，主要由兩個主要部分組成：ResidualBlock 和 SRResNet。

1. ResidualBlock:

ResidualBlock 是 SRResNet 的基本構建單元，其主要功能是學習輸入與輸出之間的差異。

結構：

- 兩個卷積層 (conv1 和 conv2)：每層使用 3x3 的卷積核，保持通道數不變。
- 兩個批次正規化層 (bn1 和 bn2)：用於穩定訓練過程。
- 一個 PReLU 激活函數：在第一個卷積後應用，引入非線性。

工作原理：

1. 輸入 x 首先通過第一個卷積層和批次正規化。
2. 然後經過 PReLU 激活。
3. 再通過第二個卷積層和批次正規化。
4. 最後，將原始輸入 x 加到處理後的結果上 (skip connections)。

這種結構允許網路學習殘差，有助於訓練更深的網路並緩解梯度消失問題。

2. SRResNet:

SRResNet 是整個超解析度網路的主體結構。

a. 初始特徵提取：

- conv1：9x9 卷積層，將輸入圖像轉換為 64 通道的特徵圖。
- prelu：引入非線性。

b. 深度特徵提取：

- residual_blocks：16 個連續的 ResidualBlock。
- conv2 和 bn1：額外的卷積和批次正規化層。
- 全局殘差學習：初始特徵與深度特徵相加。

c. 上採樣模塊：

- 兩個相同的上採樣階段，每個階段包含：
- 卷積層（upsample1/upsample2）：增加通道數到 256。
- PixelShuffle：將特徵圖的空間尺寸放大 2 倍。
- PReLU：非線性激活。

d. 重建：

- conv3：最後的 9x9 卷積層，將特徵圖轉換回 RGB 圖像。

工作流程：

1. 輸入低解析度圖像首先通過初始特徵提取。
2. 然後經過一系列殘差塊進行深度特徵提取。
3. 應用全局殘差連接。
4. 通過兩個上採樣階段將特徵圖放大 4 倍。
5. 最後重建得到高解析度輸出圖像。

How I improve PSNR?

Data Argumentation:

```
# data argumentation
self.train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)
])
```

1. transforms.RandomHorizontalFlip(): 隨機水平翻轉圖像。
2. transforms.RandomVerticalFlip(): 隨機垂直翻轉圖像。
3. transforms.RandomRotation(30): 隨機旋轉圖像，角度範圍為 [-30, 30] 度。
4. transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1): 隨機調整圖像的亮度、對比度、飽和度和色調。

Optimizer:

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4, weight_decay = 1e-5)
```

1. 自適應學習率：

Adam 為每個參數計算自適應的學習率。

它根據梯度的一階矩估計（均值）和二階矩估計（未中心化的方差）來調整每個參數的學習率。

2. 動量：

Adam 使用動量的概念，幫助優化器在相關方向上加速，同時在不相關方向上減速。

這有助於更快地收斂，並減少震盪。

3. 偏差修正：

Adam 實現了偏差修正，這有助於在訓練初期更準確地估計矩。

Hyperparameter Adjusting:

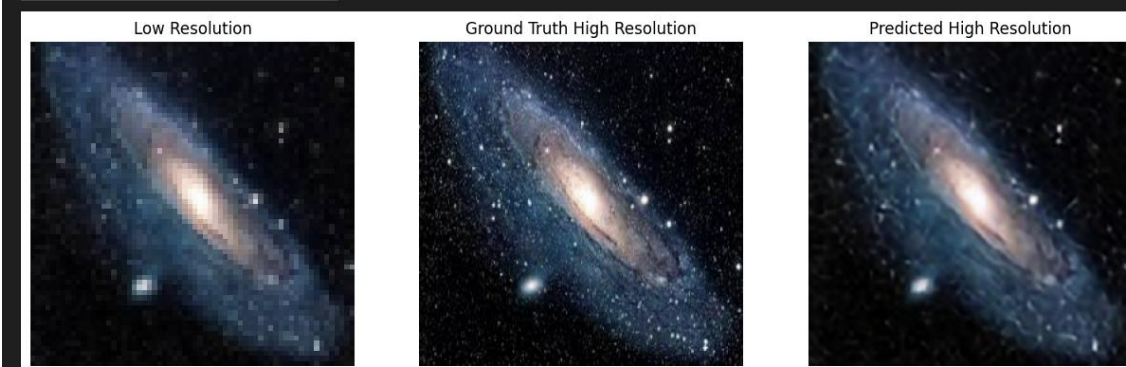
Epoch : 300, batch size : 4。

Validation PSNR :

Epoch 271	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.17 dB
Epoch 272	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.17 dB
Epoch 273	Train Loss: 0.0014	Val Loss: 0.0034	PSNR: 24.85 dB
Epoch 274	Train Loss: 0.0015	Val Loss: 0.0032	PSNR: 25.12 dB
Epoch 275	Train Loss: 0.0015	Val Loss: 0.0031	PSNR: 25.20 dB
Epoch 276	Train Loss: 0.0014	Val Loss: 0.0034	PSNR: 24.84 dB
Epoch 277	Train Loss: 0.0015	Val Loss: 0.0034	PSNR: 24.80 dB
Epoch 278	Train Loss: 0.0014	Val Loss: 0.0034	PSNR: 24.89 dB
Epoch 279	Train Loss: 0.0014	Val Loss: 0.0034	PSNR: 24.85 dB
Epoch 280	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 24.93 dB
Epoch 281	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 25.03 dB
Epoch 282	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 24.91 dB
Epoch 283	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 25.01 dB
Epoch 284	Train Loss: 0.0014	Val Loss: 0.0031	PSNR: 25.28 dB
Epoch 285	Train Loss: 0.0015	Val Loss: 0.0034	PSNR: 24.91 dB
Epoch 286	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 24.95 dB
Epoch 287	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 24.96 dB
Epoch 288	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.13 dB
Epoch 289	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.10 dB
Epoch 290	Train Loss: 0.0013	Val Loss: 0.0032	PSNR: 25.07 dB
Epoch 291	Train Loss: 0.0013	Val Loss: 0.0033	PSNR: 25.03 dB
Epoch 292	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 24.96 dB
Epoch 293	Train Loss: 0.0014	Val Loss: 0.0035	PSNR: 24.78 dB
Epoch 294	Train Loss: 0.0015	Val Loss: 0.0033	PSNR: 24.99 dB
Epoch 295	Train Loss: 0.0014	Val Loss: 0.0035	PSNR: 24.73 dB
Epoch 296	Train Loss: 0.0014	Val Loss: 0.0031	PSNR: 25.26 dB
Epoch 297	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.07 dB
Epoch 298	Train Loss: 0.0014	Val Loss: 0.0033	PSNR: 25.01 dB
Epoch 299	Train Loss: 0.0014	Val Loss: 0.0032	PSNR: 25.16 dB
Epoch 300	Train Loss: 0.0015	Val Loss: 0.0036	PSNR: 24.56 dB

Task1 Testing PSNR: 25.34dB

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.06748828..1.0290797].
PSNR for the testing data: 25.34 dB



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.05865439..1.063497].



● Task 2

```
class RRDBNet(nn.Module):
    def __init__(self, in_channels, out_channels, num_features, num_blocks, num_grow_channels=32):
        super(RRDBNet, self).__init__()
        self.conv_first = nn.Conv2d(in_channels, num_features, 3, 1, 1)
        self.body = nn.Sequential(*[RRDB(num_features, num_grow_channels) for _ in range(num_blocks)])
        self.conv_body = nn.Conv2d(num_features, num_features, 3, 1, 1)
        self.conv_up1 = nn.Conv2d(num_features, num_features, 3, 1, 1)
        self.conv_up2 = nn.Conv2d(num_features, num_features, 3, 1, 1)
        self.conv_hr = nn.Conv2d(num_features, num_features, 3, 1, 1)
        self.conv_last = nn.Conv2d(num_features, out_channels, 3, 1, 1)
        self.lrelu = nn.LeakyReLU(negative_slope=0.2, inplace=True)

    def forward(self, x):
        feat = self.conv_first(x)
        body_feat = self.conv_body(self.body(feat))
        feat = feat + body_feat
        feat = self.lrelu(self.conv_up1(F.interpolate(feat, scale_factor=2, mode='nearest'))))
        feat = self.lrelu(self.conv_up2(F.interpolate(feat, scale_factor=2, mode='nearest'))))
        out = self.conv_last(self.lrelu(self.conv_hr(feat)))
        return out

class RRDB(nn.Module):
    def __init__(self, num_features, num_grow_channels):
        super(RRDB, self).__init__()
        self.rdb1 = RDB(num_features, num_grow_channels)
        self.rdb2 = RDB(num_features, num_grow_channels)
        self.rdb3 = RDB(num_features, num_grow_channels)

    def forward(self, x):
        out = self.rdb1(x)
        out = self.rdb2(out)
        out = self.rdb3(out)
        return out * 0.2 + x

class RDB(nn.Module):
    def __init__(self, num_features, num_grow_channels):
        super(RDB, self).__init__()
        self.conv1 = nn.Conv2d(num_features, num_grow_channels, 3, 1, 1)
        self.conv2 = nn.Conv2d(num_features + num_grow_channels, num_grow_channels, 3, 1, 1)
        self.conv3 = nn.Conv2d(num_features + 2 * num_grow_channels, num_grow_channels, 3, 1, 1)
        self.conv4 = nn.Conv2d(num_features + 3 * num_grow_channels, num_features, 3, 1, 1)
        self.lrelu = nn.LeakyReLU(negative_slope=0.2, inplace=True)

    def forward(self, x):
        x1 = self.lrelu(self.conv1(x))
        x2 = self.lrelu(self.conv2(torch.cat((x, x1), 1)))
        x3 = self.lrelu(self.conv3(torch.cat((x, x1, x2), 1)))
        x4 = self.lrelu(self.conv4(torch.cat((x, x1, x2, x3), 1)))
        return x4 * 0.2 + x
```

我所使用的是 ESRGAN (Enhanced Super-Resolution Generative Adversarial Network)，以上網路架構是 ESRGAN 其中得核心部分 RRDBNet (Residual in Residual Dense Block Network)，其中有三個主要的部份。

1. RRDBNet：

這是整個網路的主體結構。它主要由以下部分組成：

- 初始卷積層 (conv_first)：將輸入圖像轉換為特徵圖。
- 主體部分 (body)：由多個 RRDB 塊組成，是網路的核心。

- 上採樣部分 (conv_up1, conv_up2)：將特徵圖的分辨率提高 4 倍。
- 最終輸出層 (conv_last)：將處理後的特徵圖轉換回圖像。

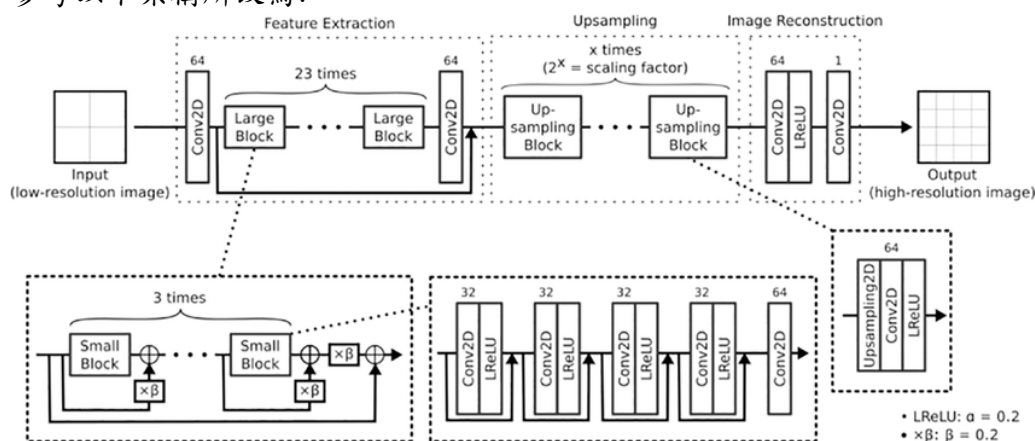
2. RRDB (Residual in Residual Dense Block) :

- 這是網絡的核心構建塊，每個 RRDB 包含三個 RDB。
- 它通過串聯三個 RDB 來處理特徵。
- 最後應用一個縮放因子 (0.2) 並加上輸入，形成殘差連接。

3. RDB 類 (Residual Dense Block) :

- 這是 RRDB 的基本單元，designed 以提取豐富的局部特徵。
- 包含四個卷積層，每層的輸出都與之前的所有層級聯。
- 使用 LeakyReLU 作為激活函數。
- 最後一層將特徵壓縮回原始通道數，並應用縮放因子和殘差連接。

參考以下架構所改寫：



Source: https://www.researchgate.net/figure/The-architecture-of-the-ESRGAN-generator-8-ie-the-deep-neural-network-used-in-the_fig4_342616974

另外 ESRGAN 在 Lossfunction 上也有改變：

```
# Perceptual Loss
class VGGLoss(nn.Module):
    def __init__(self):
        super(VGGLoss, self).__init__()
        vgg = models.vgg19(pretrained=True).features[:35].eval()
        for param in vgg.parameters():
            param.requires_grad = False
        self.vgg = vgg.to(device)
        self.criterion = nn.MSELoss()

    def forward(self, sr, hr):
        sr_feat = self.vgg(sr)
        hr_feat = self.vgg(hr)
        return self.criterion(sr_feat, hr_feat)

# Loss functions
content_criterion = nn.L1Loss()
perceptual_criterion = VGGLoss()
```

1. 感知損失 (Perceptual Loss)：

使用預訓練的 VGG 網絡提取特徵，計算生成圖像和真實圖像在高層特徵空間的差異。

2. 對抗損失 (Adversarial Loss)：

採用相對對抗損失，使生成器不僅要欺騙判別器，還要使生成的圖像比真實圖像"更真實"。

3. 內容損失 (Content Loss)：

在像素空間使用 L1 損失，確保生成的圖像在整體結構上與真實圖像一致。

ESRGAN 與 SRResNet 的比較

1. 更深和更複雜的網絡結構：

- ESRGAN 使用 RRDB 塊，而 SRResNet 使用簡單的殘差塊。
- RRDB 塊能夠學習更複雜的特徵表示，有助於重建更細緻的紋理。

2. 去除批量正規化：

- SRResNet 使用批量正規化，而 ESRGAN 移除了這些層。
- 這使得 ESRGAN 能夠保留更多的圖像細節和對比度訊息。

3. 對抗訓練：

- ESRGAN 採用 GAN 框架，而 SRResNet 僅使用像素級損失。
- GAN 框架使得 ESRGAN 能夠生成更逼真、更自然的圖像。

4. 感知損失：

- ESRGAN 使用 VGG 特徵的感知損失，而 SRResNet 主要依賴像素級 MSE 損失。

- 感知損失有助於生成在視覺上更令人滿意的結果。

Validation PSNR:

```

Model saved
Epoch 1 Train Loss: 0.5192 | Val Loss: 0.0949 | PSNR: 17.07 dB
Model saved
Epoch 2 Train Loss: 0.3741 | Val Loss: 0.0806 | PSNR: 18.56 dB
Model saved
Epoch 3 Train Loss: 0.3221 | Val Loss: 0.0713 | PSNR: 19.43 dB
Model saved
Epoch 4 Train Loss: 0.2958 | Val Loss: 0.0553 | PSNR: 21.02 dB
Epoch 5 Train Loss: 0.2789 | Val Loss: 0.0603 | PSNR: 20.87 dB
Model saved
Epoch 6 Train Loss: 0.2706 | Val Loss: 0.0551 | PSNR: 21.44 dB
Model saved
Epoch 7 Train Loss: 0.2607 | Val Loss: 0.0493 | PSNR: 22.03 dB
Epoch 8 Train Loss: 0.2577 | Val Loss: 0.0565 | PSNR: 21.39 dB
Model saved
Epoch 9 Train Loss: 0.2521 | Val Loss: 0.0508 | PSNR: 22.24 dB
Model saved
Epoch 10 Train Loss: 0.2453 | Val Loss: 0.0497 | PSNR: 22.39 dB
Model saved
Epoch 11 Train Loss: 0.2381 | Val Loss: 0.0473 | PSNR: 22.44 dB
Epoch 12 Train Loss: 0.2351 | Val Loss: 0.0520 | PSNR: 22.13 dB
Epoch 13 Train Loss: 0.2377 | Val Loss: 0.0516 | PSNR: 22.24 dB
Model saved
Epoch 14 Train Loss: 0.2301 | Val Loss: 0.0485 | PSNR: 22.45 dB
Epoch 15 Train Loss: 0.2252 | Val Loss: 0.0496 | PSNR: 22.44 dB
...
Epoch 97 Train Loss: 0.0984 | Val Loss: 0.0387 | PSNR: 24.01 dB
Epoch 98 Train Loss: 0.0975 | Val Loss: 0.0383 | PSNR: 24.01 dB
Epoch 99 Train Loss: 0.0994 | Val Loss: 0.0389 | PSNR: 23.97 dB
Epoch 100 Train Loss: 0.0980 | Val Loss: 0.0383 | PSNR: 24.03 dB
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Testing PSNR: 24.00dB

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.16860972..1.2060795].
PSNR for the testing data: 24.00 dB



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.15610743..1.108395].



● Explain what is PixelShuffle

PixelShuffle 是一種用於超分辨率任務的上採樣技術，它的主要目的是將低分辨率的特徵圖高效地轉換為高分辨率的輸出。

工作原理：

- PixelShuffle 接收一個形狀為 $(*, C * r^2, H, W)$ 的張量作為輸入，其中 r 是放大因子。
- 它將這個張量重新排列成 $(*, C, H * r, W * r)$ 的形狀。
- 具體來說，它將通道維度的像素重新分配到空間維度，從而增加圖像的高度和寬度。

舉例說明：

假設我們有一個 $(1, 16, 32, 32)$ 的特徵圖，放大因子 $r = 2$ ：

1. 輸入張量形狀： $(1, 16, 32, 32)$
2. PixelShuffle 操作後： $(1, 4, 64, 64)$

優點：

1. 計算效率高：相比於反卷積（transposed convolution），PixelShuffle 不需要學習額外的參數。
2. 避免棋盤效應：反卷積容易產生的棋盤狀偽影在 PixelShuffle 中不會出現。
3. 保留空間相關性：它保留了低分辨率特徵圖中的空間訊息。

在超分辨率網絡中的應用：

- 通常在網絡的最後階段使用，將深層特徵轉換為高分辨率輸出。
- 常見做法是先用卷積層增加通道數，然後使用 PixelShuffle 進行上採樣。

Source: <https://blog.csdn.net/g11d111/article/details/82855946>

● PSNR explained and its limitations:

PSNR 是衡量重建圖像質量的常用指標，單位為分貝（dB）。它通過計算原始圖像和重建圖像之間的均方誤差（MSE）來定義：

$$\text{PSNR} = 10 * \log_{10}((\text{MAX}^2) / \text{MSE})$$

其中 MAX 是像素的最大可能值（對於 8 位圖像為 255）。

為什麼 PSNR 不是唯一使用的指標：

1. 僅關注像素級差異：
 - PSNR 只測量重建圖像與原始圖像在像素層面的差異。
 - 它無法捕捉人類視覺系統感知的所有方面，如結構訊息或紋理細節。
2. 對全局亮度變化敏感：
 - 即使圖像在視覺上相似，輕微的全局亮度變化也可能導致 PSNR 值大幅下降。

3. 不考慮人眼感知：

- 人眼對某些類型的失真比其他類型更敏感，但 PSNR 對所有像素差異一視同仁。

4. 缺乏結構訊息：

- PSNR 不考慮圖像的結構特徵，這對於人類感知圖像質量至關重要。

5. 在高質量範圍不夠敏感：

- 當圖像質量很高時，PSNR 的變化可能不足以反映肉眼可見的差異。

以下是不同視角的圖像質量評估指標：

1. 結構相似性指數 (SSIM)：

- 考慮亮度、對比度和結構訊息。
- 更接近人類視覺系統的感知。
- 取值範圍為 -1 到 1，1 表示完全相同。

2. 多尺度結構相似性指數 (MS-SSIM)：

- SSIM 的擴展版本，在多個尺度上評估圖像相似性。
- 更好地捕捉不同尺度的視覺特徵。

3. 特徵相似性 (FSIM)：

- 使用相位一致性和梯度幅度來評估圖像質量。
- 更好地捕捉邊緣和紋理訊息。

4. 視覺訊息保真度 (VIF)：

- 基於自然場景統計和人類視覺系統模型。
- 評估訊息保留程度，而不僅僅是失真。

5. 感知評估指標 (LPIPS)：

- 使用深度學習模型來評估圖像相似性。
- 更接近人類對圖像差異的判斷。

6. 失真指數 (IFC)：

- 測量從參考圖像到失真圖像的相互訊息。
- 考慮人類視覺系統的特性。

7. 噪聲質量測量 (NQM)：

- 考慮人類視覺系統對不同類型噪聲的敏感度。
- 特別適用於評估有噪聲的圖像。

Source: <https://adam-study-note.medium.com/%E9%9B%BB%E8%85%A6%E8%A6%96%E8%A6%BA-%E5%9C%96%E5%83%8F%E8%A1%A1%E9%87%8F%E6%8C%87%E6%A8%99psnr-ssim%E4%BB%8B%E7%B4%B9-a4fd76d9d84d>